LifeBridge: A Smart Organ Transplant Coordination System

A. Nikitha
Department of Computer Science and Engineering
KLH CSE, Bowrampet Campus, India

V. Sraavya
Department of Computer Science and Engineering
KLH CSE, Bowrampet Campus, India

G. Rithwik
Department of Computer Science and Engineering
KLH CSE, Bowrampet Campus, India

Project guide: Venkateswari Ketineni, Assistant Professor

Abstract

Organ transplantation requires coordination among donors, recipients, hospitals, transplant centers, medical teams, and authorized personnel. Disconnected records and manual communication can complicate the maintenance of donor and recipient information, organ availability, compatibility review, allocation, and notifications. This paper presents LifeBridge, a smart organ transplant coordination platform designed around service-oriented architecture (SOA) and a proposed microservices workflow. The project materials describe a responsive browser-based frontend with dashboard statistics, donor and recipient management, organ inventory, matching and allocation views, notifications, and a service-control center. The intended architecture separates donor, recipient, organ, matching and allocation, notification, and authentication functions, with REST APIs and planned API Gateway and Eureka service discovery. The supplied project review states that the frontend prototype and sample datasets have been demonstrated, while backend services, database integration, authentication, and advanced matching integration remain planned. A dashboard screenshot displays demo-network indicators including seven active organs, six urgent recipients, six pending matches, 42 allocations this month, 12 participating centers, and an 88.4% readiness index. These are interface demo values, not independently measured operational outcomes. The paper documents the project's current scope and architecture without claiming validated clinical matching or real-world transplant performance.

Index Terms—organ transplantation, healthcare information systems, Spring Boot, REST API, role-based access, organ availability

I. INTRODUCTION

Transplant coordination depends on accurate information being available to the right people at the right time. Donor and recipient details, hospital information, organ status, and coordination updates all need to be maintained carefully and confidentially. If these details are scattered across separate records or passed around manually, staff may not have a clear, up-to-date picture of the process.

LifeBridge brings the main coordination tasks into one interface for managing transplant-related records and workflow status. It is intended to support staff, not to make clinical decisions or replace medical professionals, official allocation rules, authorized transplant networks, or legal procedures. The current demonstration is mainly a frontend prototype populated with sample data. As stated in the project review, backend development and database integration are part of the next phase.

The project focuses on five user categories: administrators, hospitals, transplant coordinators, donors, and recipients. Each category has different responsibilities and therefore requires access to suitable functions. The main contribution of this work is the design and implementation of a role-oriented web application with a JavaScript-based frontend and Spring Boot REST services. The remainder of the paper reviews relevant system concepts, describes the architecture and workflows, defines the evaluation approach, and identifies the evidence required to report implementation results.

II. RELATED WORK

Many healthcare information systems organize records in a shared environment and present different functions to different users. This can make information easier to find and reduce repeated data entry. In workflows involving several organizations, the system also needs to separate tasks such as entering information, reviewing records, coordinating activity, and administering users. These ideas shaped the LifeBridge design.

A REST API gives the browser and the server a defined way to exchange information without combining the interface and business rules into one layer. The frontend can send requests and display the returned results, while the server remains responsible for validating data and checking permissions. Spring Boot is the planned technology for implementing LifeBridge's backend endpoints and service logic.

Privacy is especially important because donor and recipient records can include identifying and health-related details. Users should only be able to view or change information needed for their responsibilities. Logging in proves a user's identity, but it does not by itself decide what that user is allowed to do; each protected action also needs an authorization check. Secure deployment, careful logging, limited data collection, and compliance with relevant rules must be addressed as development continues.

National Organ and Tissue Transplant Organization (NOTTO) describes a national network for organ procurement, distribution, and donor/recipient registry activities in India [1]. These institutional workflows underline the need for accurate records and coordination between hospitals and authorized agencies. From the software perspective, Spring's microservices guidance describes independent services as a way to improve maintainability and resilience, while also noting the operational complexity of distributed systems [2]. Spring Cloud Gateway supports routing and cross-cutting concerns such as security, monitoring, and resilience [3], and Spring Cloud supports service discovery patterns such as Eureka [4]. These sources inform the proposed architecture; they do not establish that LifeBridge is integrated with NOTTO or that its prototype is clinically validated.

III. SYSTEM REQUIREMENTS AND DESIGN

A. User Roles

Administrator: manages system-level users and oversees records and configuration. Hospital: maintains hospital profile information and participates in organ-availability workflows. Transplant coordinator: coordinates the operational steps and monitors relevant records. Donor: submits or updates permitted donor information. Recipient: submits registration details and views permitted status information. The precise permissions must be aligned with the implemented authorization rules.

B. Functional Requirements

The system is intended to provide account registration and sign-in; role-aware dashboards; donor, recipient, and hospital registration; transplant coordinator management; organ registration; and organ-availability updates. Validation should be applied to required fields

and identifiers, and the interface should communicate successful actions and errors clearly. Records should move through explicit statuses rather than relying on ambiguous free-text notes.

C. Non-Functional Requirements

The principal non-functional requirements are confidentiality, integrity, usability, maintainability, and reliability. The backend should validate requests independently of client-side validation. Access to sensitive endpoints should be restricted by role. Errors should not expose credentials or internal implementation details. The application should also support traceable updates to important records and should be tested for invalid input, unauthorized requests, and inconsistent status transitions.

D. Scope and Safety

LifeBridge is an information-management prototype. Organ allocation, medical suitability, consent, donor-recipient compatibility, and clinical decisions must follow approved medical protocols and authorized transplant organizations. The system must not imply that a record's presence or an availability status constitutes clinical approval or legal authorization for transplantation.

IV. SYSTEM ARCHITECTURE AND METHODOLOGY

A. Architecture

The implementation follows a client-server design. The browser frontend is built with HTML, CSS, and JavaScript. It presents forms, tables, navigation, and role-specific views. The frontend communicates with Spring Boot REST endpoints using HTTP requests and exchanges structured data. Backend controllers receive requests, service components apply business rules, and persistence components manage stored records. A database is required for persistent operation; the exact database engine and schema should be stated according to the deployed implementation.

User roles: Administrator | Hospital | Coordinator | Donor | Recipient
Frontend: HTML + CSS + JavaScript ⇄ Spring Boot REST API
Backend services: Authentication / User Management / Organ Records / Availability / Coordination ⇄ Database

Fig. 1. Logical architecture of the LifeBridge system.

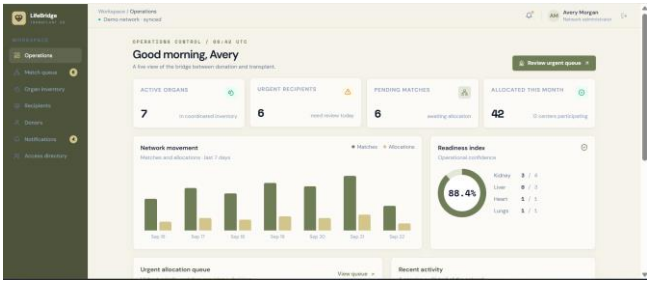


Fig. 2. LifeBridge operations dashboard prototype; displayed metrics are demo values.

B. Request Processing

A normal interaction starts when a user submits a form or opens a record. JavaScript can catch simple input problems early and then send the request to the relevant API endpoint. In the completed backend, the server must validate the submitted data, check the user's permissions, apply the required rules, and save only valid changes. The response is then shown in the interface as a message or refreshed record. Browser-side checks help usability, but they cannot replace server-side validation.

C. Core Workflows

Registration workflow: a user submits the details required for the selected role. The server validates required fields and stores the record only when validation succeeds. Record-management workflow: authorized staff retrieve and update permitted donor, recipient, or hospital records. Organ-availability workflow: an authorized role registers an organ record and updates its status according to the application's defined lifecycle. Coordination workflow: coordinators review relevant records and maintain progress information. Each workflow should record who performed a sensitive update and when it occurred.

D. Data Management

The data model should distinguish users, role-specific profiles, hospitals, organ records, and coordination/status events. Stable identifiers should link related records without duplicating personal information unnecessarily. Referential integrity and validation rules should prevent orphaned records and invalid status values. Passwords must never be stored in plaintext; an established password-hashing mechanism and secure session or token handling should be used. Sensitive data should be protected in transit and at rest where supported by the deployment environment.

V. IMPLEMENTATION

The current interface uses plain HTML, CSS, and JavaScript. HTML provides the page structure, CSS handles the layout and styling, and JavaScript supports interactions such as forms and dashboard updates. Keeping the frontend framework-free makes the client layer relatively lightweight.

The project review lists Java, REST APIs, MySQL or MongoDB, JWT authentication, Eureka service discovery, an API Gateway, and Postman among the intended technical components. The review describes the backend, database integration, authentication, and advanced matching integration as planned for a later phase. Therefore, these are presented here as the target architecture rather than as verified deployed components.

Security remains a design requirement for the planned backend. When implemented, credentials should not be stored in plaintext, sensitive endpoints should enforce server-side authorization, and logs should avoid unnecessary personal or medical details. The current project review does not provide evidence of a completed JWT implementation or security test results.

Table I summarizes the primary modules at a functional level. The exact endpoint paths, database schema, authentication mechanism, and deployment environment should be added from the source code so the description matches the implemented system.

TABLE I. PRINCIPAL SYSTEM MODULES

Module	Main responsibility	Verification focus
Authentication	Sign-in and session/token handling	Invalid credentials; protected routes
User and roles	Manage role-specific accounts	Permission enforcement
Donor/recipient	Maintain profile and registration records	Required fields; validation
Hospital/coordinator	Manage organizations and coordination	Authorized updates
Organ availability	Register records and update status	Valid status transitions

VI. RESULTS AND EVALUATION

The supplied project review documents a completed and browser-demonstrated frontend prototype, with sample datasets for donors, recipients, organs, hospitals, and allocations. It also explicitly states that backend, database integration, authentication, and advanced matching algorithm integration are planned for the next development phase. The dashboard screenshot provides a visible example of the prototype interface. No independent test report, API benchmark, or clinical validation results were supplied, so the dashboard's figures are treated as sample/demo indicators rather than measured research outcomes.

A. Functional Testing

Functional testing should cover successful and unsuccessful registration, authentication, role-specific access, creation and retrieval of records, updates to organ availability, required-field validation, and error handling. For each test, record the test identifier,

preconditions, action, expected result, observed result, and pass/fail outcome. Tests should include attempts by one role to access operations reserved for another role.

B. Quantitative Evaluation

Useful measurements include the number of test cases executed and passed, API response time under a defined test setup, error rate, and completion of key user workflows. If a dummy dataset was used, report its size and composition and clearly label it as synthetic. Performance measurements should state the hardware, software versions, network conditions, request count, and whether the reported value is a mean, median, or percentile.

C. Results Table — Populate from Actual Tests

Metric	Observed result	Evidence/source
Functional test cases passed	[Insert actual n/N]	[Test report or test log]
Role authorization tests passed	[Insert actual n/N]	[API/security test log]
Median API response time	[Insert measured value and unit]	[Benchmark output]
Dataset size	[Insert actual record count]	[Database query or dataset file]

The interface snapshot shows seven active organs, six urgent recipients, six pending matches, 42 allocations this month, 12 participating centers, and an 88.4% readiness index. The screenshot also shows a seven-day network movement chart and a readiness breakdown for kidneys, liver, heart, and lungs. These values are reproduced only as displayed demo content; the source materials do not define their calculation method or establish that they represent real transplant activity. They must not be interpreted as validated operational or clinical results.

VII. DISCUSSION

Putting workflow records in one application could make it easier for staff to find current information and understand the tasks assigned to their role. Keeping the interface separate from backend rules can also make future changes easier to manage. At this stage, these are expected benefits of the design rather than proven improvements; user feedback and implementation tests are needed to confirm them.

The main risks relate to the sensitivity and correctness of the information handled. Incorrect records, overly broad permissions, unclear status transitions, or unavailable services could disrupt coordination. A prototype should therefore be evaluated not only for successful happy-path operations but also for invalid data, unauthorized access, duplicate submissions, concurrent updates, and recovery from failures.

The current prototype has important limitations: the supplied review places backend services, database integration, authentication, and advanced matching integration in a future phase. The dashboard values are demonstrative and lack a documented data-generation or calculation method. LifeBridge is not shown to be integrated with NOTTO, and the prototype does not establish clinical compatibility or allocation validity. Further work should implement and test the backend, verify access control, document the matching logic, and review applicable transplant rules before any real patient or donor data is used.

VIII. CONCLUSION AND FUTURE WORK

This paper describes LifeBridge, a transplant-coordination project with a frontend built using HTML, CSS, and JavaScript and a planned Spring Boot REST backend. The interface is organized around five user groups—administrators, hospitals, transplant coordinators, donors, and recipients—and focuses on registration, record management, organ registration, and availability tracking. The design calls for server-side validation, role-based permissions, and a clear separation between the user interface and business logic.

The next step is to report the completed implementation's actual test outcomes, including functional coverage, authorization checks, data volume, and measured response times. Further work may include usability evaluation with representative users, audit-log review, stronger security testing, accessibility improvements, and carefully governed interoperability with authorized healthcare systems. Any clinical matching or allocation functionality would require separate validation and adherence to approved transplant policies.

ACKNOWLEDGMENT

The authors may add acknowledgments for project guidance, institutional support, or resources used. No funding or external support details were provided for this draft.

REFERENCES

[1] Directorate General of Health Services, Ministry of Health and Family Welfare, Government of India, "National Organ and Tissue Transplant Organisation (NOTTO)," [Online]. Available: https://www.dghs.mohfw.gov.in/notto.php. [Accessed: Oct. 27, 2026].
[2] VMware, Inc. and contributors, "Microservices," Spring, [Online]. Available: https://spring.io/microservices. [Accessed: Oct. 27, 2026].
[3] Broadcom, "Spring Cloud Gateway Reference Documentation," [Online]. Available: https://docs.spring.io/spring-cloud-gateway/reference/. [Accessed: Oct. 27, 2026].
[4] Broadcom, "Spring Cloud," [Online]. Available: https://spring.io/cloud. [Accessed: Oct. 27, 2026].
[5] Directorate General of Health Services, Ministry of Health and Family Welfare, Government of India, "Guidelines for Implementation of National Organ Transplant Programme," [Online]. Available: https://notto.mohfw.gov.in/writereaddata/portal/news/791_1_guidelines_04_oct_2021__approved_.pdf. [Accessed: Oct. 27, 2026].